Municipalidad de Algarrobo

Informe Técnico de Auditoría de Seguridad y Arquitectura N°2 – Plataforma “Geoportal Municipal”

Documento técnico que presenta los resultados del análisis del código fuente, evaluación de arquitectura y revisión de estándares de ciberseguridad sobre los entregables del desarrollo.

Auditoría Geoportal Municipal - SECPLAC

I. Municipalidad de Algarrobo, Departamento de Informática

Abril 2026

Índice

1. Resumen Ejecutivo 2
2. Objetivo y Metodología 3
3. Hallazgos Críticos de Seguridad y Control de Acceso 3
3.1. Filtraciones de Credenciales Críticas en Código e Historial (GitHub) 3
3.2. Principio de Menor Privilegio 3
3.3. Rutas Huérfanas 4
3.4. Configuración Permisiva de CORS y Entornos Mixtos 6
3.5. Riesgo en la Validación y Límite de Archivos Subidos 6
4. Estabilidad, Rendimiento y Concurrencia 6
4.1. Cuello de Botella y Condición de Carrera en Extracción de IA 6
4.2. Deuda Técnica en Consultas SQL (Subconsultas Correlacionadas) 7
4.3. Exposición de Claves de API de Terceros en Front-end 7
5. Arquitectura de Base de Datos y Trazabilidad 8
5.1. Colisión de Auditorías y Cascada de Triggers 8
5.2. Transaccionalidad y Duplicación en Registro de Actividades 8
5.3. Funciones y Tablas de Legado Redundantes 9
5.4. Retención de Datos en control_actividad 9
6. Infraestructura y Buenas Prácticas de Código 9
6.1. Dependencia de Almacenamiento Local (Fallback) y Portabilidad 9
6.2. Enlaces Rígidos (Hardcoding) en el Front-end 10
6.3. Borrado Físico de Usuarios (Error 500 y Riesgo de Integridad) 10
6.4. Observaciones Menores (UI/UX) 10
7. Inconsistencias en la Documentación Técnica 11
7.1. Discrepancia en el Almacenamiento de Sesiones (JWT) 11
8. Consideraciones sobre el Alcance de la Refactorización 11
9. Conclusión General 12

1. Resumen Ejecutivo

Esta segunda fase de la auditoría técnica profundizó en la arquitectura de la base de datos, el rendimiento del sistema, el análisis estático de código (SAST) y la gestión de la infraestructura. Si bien se validaron las funcionalidades básicas, se detectaron deficiencias arquitectónicas y vulnerabilidades residuales en el control de acceso y manejo de credenciales.

Se recomienda condicionar el despliegue a producción a la subsanación de los siguientes problemas identificados:

N°	Hallazgo / Vulnerabilidad	Prioridad de Resolución
3.1	Filtración de Credenciales Críticas en Código e Historial	**Bloqueante**
3.2	Principio de Menor Privilegio	**Bloqueante**
3.3	Rutas Huérfanas	**Media/Baja**
3.4	Configuración Permisiva de CORS y Entornos Mixtos	**Media**
3.5	Riesgo de Validación y Límite de Archivos Subidos	**Media**
4.1	Cuello de Botella y Condición de Carrera en Extracción de IA	**Alta**
4.2	Deuda Técnica en Consultas SQL (Subconsultas Correlacionadas)	**Media/Baja**
4.3	Exposición de Claves de API de Terceros en Frontend	**Alta**
5.1	Colisión de Auditorías y Cascada de Triggers	**Media**
5.2	Transaccionalidad y Duplicación de Registro de Actividades	**Alta**
5.3	Funciones y Tablas de Legado Redundantes	**Baja**
5.4	Retención de Datos en control_actividad	**Media/Baja**
6.1	Dependencia de Almacenamiento Local (Fallback)	**Alta**
6.2	Enlaces Rígidos (Hardcoding de URL) en el Frontend	**Media**
6.3	Borrado Físico de Usuarios (Error 500 y Riesgo de Integridad)	**Media**
6.4	Observaciones Menores (UI/UX)	**Media**
7.1	Discrepancia en el Almacenamiento de Sesiones (JWT)	**Baja**

- Bloqueante: debe corregirse antes de recepción conforme.
- Alta: debe corregirse con prioridad en el corto plazo.
- Media: planificable, no impide recepción si existe compromiso/plan.
- Baja: planificable, no impide recepción si existe compromiso/plan.

2. Objetivo y Metodología

El presente documento tiene como propósito visibilizar las deudas técnicas, fallas arquitectónicas y vulnerabilidades críticas presentes en la plataforma entregada, las cuales deben subsanarse antes de que el sistema sea autorizado para operar en un entorno de producción institucional.

- **Alcance:** La auditoría consistió en una revisión exhaustiva del código fuente (Frontend y Backend), en esta ocasión resultó bastante más necesaria la inspección manual del repositorio ya que con análisis estático automatizado (Bandit y Semgrep) no se logró detectar grandes problemas.
- **Limitaciones:** La revisión se restringió a la evidencia disponible en el repositorio correspondiente al enlace entregado y no constituye una prueba de penetración dinámica (Pentesting) sobre la infraestructura final.

  https://geoportalalgarrobo.github.io/ALGARROBO_BASE/frontend/index.html

  https://github.com/geoportalalgarrobo/ALGARROBO_BASE

3. Hallazgos Críticos de Seguridad y Control de Acceso

3.1. Filtraciones de Credenciales Críticas en Código e Historial (GitHub)

- **Observación:** Se identificaron valores sensibles en el código como fallback de variables de entorno, incluyendo **JWT_SECRET** y una cadena de conexión completa **DB_CONNECTION_STRING** (con credenciales) en config.py. Adicionalmente, según revisión del historial del repositorio, habría existido exposición accidental de un archivo .env en commits (ej. **c4a595d**, **e7221dd**).
- **Impacto:** Alto riesgo de compromiso. Si el repositorio (o su historial) es accesible a terceros, un actor podría obtener secretos y credenciales, habilitando acceso no autorizado a la base de datos y/o falsificación/validación indebida de sesiones JWT (dependiendo de vigencia y alcance de los secretos).
- **Recomendación:** Rotar todas las credenciales y secretos criptográficos antes de cualquier paso a producción. Se debe purgar el historial completo del repositorio (utilizando herramientas como **git filter-repo**) para borrar cualquier rastro de archivos **.env** antiguos. Las variables sensibles deben gestionarse exclusivamente mediante el entorno del servidor y nunca escribirse en el código. Tampoco se recomienda usar valores fallback para secretos del servidor, es preferible que el sistema falle en producción antes que usar valores por defecto.

3.2. Principio de Menor Privilegio

- **Observación:** El sistema delega parte relevante del control de acceso al navegador del cliente mediante lógica en **router.js**, validando sesión/rol en base a valores almacenados en **localStorage** (token, flags y userData) y

permitiendo únicamente los niveles 10 y 11 a nivel de flujo UI (router.js). Sin embargo, este control no constituye una barrera de seguridad real. En el Backend, el decorador **@admin_required** valida permisos consultando a la base de datos, pero considera "administrador" a cualquier usuario con **nivel_acceso >= 10** (decorators.py), lo que incluye a **admin_proyectos (11)**. Adicionalmente, existen endpoints críticos de mutación que solo exigen autenticación (**@session_required**) y no aplican un control de rol/perfil más estricto, por ejemplo: [POST] /api/proyectos, [PUT] /api/proyectos/<pid>, [DELETE] /api/proyectos/<pid> (proyectos_routes.py).

- **Impacto:** Se produce una falta de segregación de funciones entre roles administrativos. En el Frontend, un usuario con sesión válida puede acceder por URL a vistas administrativas aún cuando no correspondan a su rol, debido a que el control es principalmente de flujo/cliente y no una restricción robusta del lado del servidor (route.js). A nivel de servidor, el riesgo es mayor: **admin_proyectos (11)** hereda permisos administrativos porque **@admin_required** acepta cualquier **nivel_acceso >= 10**, habilitándole acciones de administración que deberían ser exclusivas de **admin_general (10)**, como la gestión de usuarios vía endpoints protegidos con **@admin_required** (por ejemplo, creación/edición/eliminación) (users_routes.py). En consecuencia, el sistema no garantiza el principio de menor privilegio ni la separación entre "gestión de proyectos" y "administración del sistema".

- **Recomendación:** Aplicar Defensa en Profundidad en ambas capas:

  1. **En el Frontend (Control de Flujo):** Endurecer la lógica de **router.js** para que el control de rutas sea estricto (lista blanca efectiva) y evitar que **admin_proyectos (11)** pueda navegar/renderizar vistas reservadas a **admin_general (10)** (fragmento /admin_general/).

  2. **En el Backend (Seguridad Real):** La autorización no debe depender del Frontend. Reemplazar el enfoque **nivel >= 10** por un control explícito por rol (ej. **@role_required(allowed_roles)** o equivalente), permitiendo definir exactamente qué roles pueden ejecutar cada endpoint. Aplicar este control de forma consistente en rutas de administración y en endpoints críticos de mutación (crear/editar/eliminar), alineando permisos con las responsabilidades reales de **admin_general (10)** vs **admin_proyectos (11)**.

3.3. Rutas Huérfanas

- **Observación:** El Frontend incluye múltiples páginas HTML accesibles vía URL directa que no forman parte del flujo de navegación principal o representan integración parcial/inconsistente (listado al final de este punto). Este listado se mantiene como inventario para revisión de alcance, no como hallazgo crítico de seguridad por sí mismo.

- **Impacto:** Mantener rutas/módulos no integrados o inconsistentes incrementa la superficie mantenimiento y puede generar confusión sobre cual es el flujo operativo oficial.

- **Recomendación:** No se exige eliminarlas de inmediato. Se recomienda revisar si serán parte del flujo, si requieren integrarse formalmente a la navegación, y si deben retirarse del despliegue si quedan fuera del alcance de proyecto.

    - frontend/division/transparencia > toda la rama hacia abajo
    - frontend/division/seguridad > toda la rama hacia abajo

    - frontend/ayuda.html
    - frontend/buscador.html
    - frontend/documento.html
    - frontend/file.html
    - frontend/fiscalizacion.html
    - frontend/notificaciones.html
    - frontend/perfil.html
    - frontend/vizualizar.html

    - frontend/administracion/auditoria.html
    - frontend/administracion/divisiones.html
    - frontend/administracion/proyectos.html
    - frontend/administracion/sistema.html

    - frontend/division/licitaciones/admin_general/dashboard.html
    - frontend/division/licitaciones/admin_proyectos/seguimiento.html
    - frontend/division/licitaciones/director_obras/dashboard.html

    - frontend/division/secplan/admin_general/analisis.html
    - frontend/division/secplan/admin_general/calendario.html
    - frontend/division/secplan/admin_general/documento.html
    - frontend/division/secplan/admin_general/geomapas.html
    - frontend/division/secplan/admin_general/header.html
    - frontend/division/secplan/admin_general/hitos.html
    - frontend/division/secplan/admin_general/informe.html
    - frontend/division/secplan/admin_general/informe_dinamico.html
    - frontend/division/secplan/admin_general/informe_dinamico2.html
    - frontend/division/secplan/admin_general/mapa.html
    - frontend/division/secplan/admin_general/mapa2.html
    - frontend/division/secplan/admin_general/observacion.html
    - frontend/division/secplan/admin_general/user.html
    - frontend/division/secplan/admin_general/vecinos.html

    - frontend/division/secplan/admin_proyectos/analisis.html
    - frontend/division/secplan/admin_proyectos/calendario.html
    - frontend/division/secplan/admin_proyectos/dashboard.html
    - frontend/division/secplan/admin_proyectos/documento.html
    - frontend/division/secplan/admin_proyectos/header.html
    - frontend/division/secplan/admin_proyectos/informe.html
    - frontend/division/secplan/admin_proyectos/mapa.html
    - frontend/division/secplan/admin_proyectos/proyecto.html

    - frontend/division/secplan/director_obras/analisis.html
    - frontend/division/secplan/director_obras/calendario.html
    - frontend/division/secplan/director_obras/dashboard.html
    - frontend/division/secplan/director_obras/header.html
    - frontend/division/secplan/director_obras/informe.html
    - frontend/division/secplan/director_obras/mapa.html
    - frontend/division/secplan/director_obras/proyecto.html

3.4. Configuración Permisiva de CORS y Entornos Mixtos

- **Observación:** Si la variable de entorno **ALLOWED_ORIGINS** no está definida, el sistema aplica un fallback permisivo **ALLOWED = ["*"]** (config.py). Esta lista se usa para configurar CORS globalmente con **supports_credentials=True** (app_railway.py).
- **Impacto:** Una configuración permisiva de CORS aumenta la exposición de la API a consumo desde orígenes no previstos. Si la API utiliza credenciales o tokens accesibles desde el navegador, esto puede facilitar escenarios de abuso (por ejemplo, integraciones no autorizadas o automatización desde sitios externos). Además, el uso de fallbacks dificulta asegurar que producción y desarrollo tengan políticas distintas y verificables.
- **Recomendación:** Eliminar fallbacks permisivos en código para entornos productivos. Definir **ALLOWED_ORIGINS** de forma explícita por entorno (Desarrollo/Producción) y exigir valores estrictos en producción (lista blanca), dejando el modo permisivo solo como opción controlada y explícita para desarrollo/pruebas.

3.5. Riesgo en la Validación y Límite de Archivos Subidos

- **Observación:** La validación de archivos subidos es débil. La función **allowed_file()** no aplica una lista blanca real; acepta cualquier archivo mientras tenga extensión (presencia de ".") (audit_logger.py). Si bien existe **ALLOWED_EXTENSIONS**, su uso actual es para acortar la extracción de texto/OCR (no para bloquear subidas) (config.py, documentos_routes.py). Adicionalmente, el Backend define **MAX_CONTENT_LENGTH** en 1GB (comentado como "migración ZIP"), lo que resulta permisivo para operación normal (app_railway.py).
- **Impacto:** Permite subir archivos no esperados o potencialmente maliciosos renombrados bajo extensiones arbitrarias y aumenta el riesgo de consumo excesivo de recursos (almacenamiento/CPU) por archivos grandes o por volumen de subidas.
- **Recomendación:** Implementar una política de subida más estricta en el Backend: (1) lista blanca efectiva de tipos permitidos (extensión + validación de MIME/"magic bytes" según corresponda), (2) límites de tamaño acotados y parametrizados por entorno (migración/desarrollo vs producción), y (3) mantener el límite en Nginx como capa adicional, sin depender únicamente del proxy.

4. Estabilidad, Rendimiento y Concurrencia

4.1. Cuello de Botella y Condición de Carrera en Extracción de IA

- **Observación:** El sistema expone un endpoint que retorna el texto extraído de los documentos de un proyecto y lo calcula en tiempo real, iterando archivos y

ejecutando parse/OCR "al vuelo" (documentos_routes.py). Para archivos **.doc**, la extracción se realiza mediante **subprocess.run()** invocando LibreOffice en modo headless, de forma síncrona, y el texto resultante se identifica tomando el archivo **.txt** "más reciente" dentro del directorio del proyecto (extract.py).

- **Impacto:** La conversión/lectura es una operación pesada y síncrona que puede bloquear el worker que atiende la solicitud, degradando el rendimiento bajo concurrencia (especialmente si existen muchos documentos o documentos **.doc**). Además, el paso de seleccionar el **.txt** "más reciente" en una carpeta compartida introduce una condición de carrera: múltiples conversiones concurrentes dentro del mismo directorio podrían provocar cruces de contenido (un request leyendo el **.txt** generado por otro).
- **Recomendación:** Evitar extraer el texto durante la consulta del chatbot. Preprocesar la extracción al momento de la carga del documento (o mediante un job asíncrono), persistiendo el texto resultante (por archivo) y reusándolo en consultas posteriores. Para eliminar la condición de carrera, usar un output determinístico por archivo (o un directorio temporal por ejecución) en vez de "el más reciente". Como mejora, si el volumen de texto supera límites prácticos de contexto, incorporar un esquema tipo RAG (fragmentación + recuperación) para enviar solo fragmentos relevantes para la consulta.

4.2. Deuda Técnica en Consultas SQL (Subconsultas Correlacionadas)

- **Observación:** En los endpoints de listado de proyectos se calcula el campo **ult_modificacion** mediante **GREATEST(...)** con múltiples subconsultas correlacionadas **SELECT MAX(...) ... WHERE proyecto_id = p.id** (hitos, observaciones, documentos y geomapas). Esta lógica se repite en **[GET] /proyectos4**, **[GET] /proyectos_chat** y **[GET] /proyectos** (proyectos_routes.py).
- **Impacto:** El cálculo de máximos por tablas relacionadas puede volver la consulta costosa a medida que aumenta el volumen de proyectos y registros asociados, degradando progresivamente tiempos de respuesta y aumentando la carga sobre la base de datos.
- **Recomendación:** Consolidar la "última modificación" en **proyectos.fecha_actualizacion** y mantenerla actualizada mediante **Triggers** en DB cuando cambien entidades relacionadas (hitos, observaciones, documentos, geomapas). Con esto se evita recalcular máximos de cada request y se simplifican los listados.

4.3. Exposición de Claves de API de Terceros en Frontend

- **Observación:** El módulo de Chatbot realiza la llamada al proveedor de IA directamente desde el navegador y envía la credencial en el header **Authorization: Bearer ${API_KEY}** (chat.html).
- **Impacto:** La clave queda expuesta al cliente (aunque esté ofuscada), pudiendo ser extraída y reutilizada para consumo no autorizado del servicio. Si

bien esto no es tan grave en la situación actual, al tratarse de un servicio gratuito, si impide contratar un servicio de IA de pago a futuro si se mantiene esta configuración.

- **Recomendación:** Implementar un patrón Backend Proxy: el Frontend envía la consulta a un endpoint; el Backend resguarda la clave en variables de entorno, ejecuta la llamada al proveedor, aplica controles (rate-limit, logging, cuotas) e integra el contexto (RAG si aplica) antes de responder al cliente.

5. Arquitectura de Base de Datos y Trazabilidad

5.1. Colisión de Auditorías y Cascada de Triggers

- **Observación:** Al crear un proyecto se registran dos eventos ("creación" y luego "edición") por efecto de cascada: un trigger **AFTER INSERT ON proyectos** inserta un hito automático y luego un trigger **AFTER INSERT ON proyectos_hitos** actualiza el proyecto, disparando nuevamente el trigger de auditoría sobre **proyectos** hacia **control_actividad**.
- **Impacto:** Ruido/duplicación en la trazabilidad y aumento innecesario de escrituras (bloat), lo que también incrementa el costo de retención descrito en el punto 5.4.
- **Recomendación:** Revisar la lógica de triggers/auditoría para evitar que efectos internos (cascadas) generen eventos equivalentes a acciones del usuario cuando no corresponden.

5.2. Transaccionalidad y Duplicación en Registro de Actividades

- **Observación:** En operaciones CRUD se mezcla auditoría por trigger con auditoría desde aplicación. Por ejemplo, en **[DELETE] /proyectos/<pid>** se ejecuta la operación **UPDATE + COMMIT** y posteriormente se registra actividad vía **log_control()** fuera de la transacción (proyectos_routes.py, audit_logger.py).
- **Impacto:** Riesgo de desincronización (operación confirmada sin registro, o registro duplicado/inconsistente) y aumento del volumen de escritura en **control_actividad** (relacionado con 5.4).
- **Recomendación:** Definir el registro transaccional de actividad para CRUD en un solo mecanismo (idealmente triggers en DB para que quede atómico), y reservar **log_control()** para eventos no transaccionales o de capa web (login, descargas, etc). Si se requiere conservar campos de contexto web (ip, user_agent, endpoint) en registros generados por triggers, definir un mecanismo para propagar ese contexto a la sesión de DB antes de ejecutar el DML.

5.3. Funciones y Tablas de Legado Redundantes

- **Observación:** Coexisten múltiples mecanismos/tablas de auditoría. A nivel de aplicación, **log_auditoria()** registra en **control_actividad** y además inserta un registro legacy en **auditoria** (audit_logger.py). A nivel de API, existe un endpoint legacy que consulta **auditoria** (**/api/auditoria**) (calendario_routes.py). Adicionalmente existe la tabla **auditoria2** definida en scripts SQL, que no es consumida por ningún endpoint.
- **Impacto:** Redundancia y fragmentación de la trazabilidad (doble escritura) que incrementa bloat y complejidad operativa.
- **Recomendación:** Revisar y consolidar explícitamente que tabla es operativa y cuáles son legado. Evaluar la eliminación de tablas que no sean objetivamente funcionales para la puesta en marcha final del proyecto.

5.4. Retención de Datos en control_actividad

- **Observación:** La tabla **control_actividad** almacena el JSON completo de **datos_antes** y **datos_despues** de cada registro.
- **Impacto:** Genera registros pesados y crecimiento sostenido del almacenamiento, especialmente si coexisten dobles escrituras o eventos duplicados.
- **Recomendación:** Aislar la tabla (quitar cualquier relación estricta con otras tablas) para facilitar, a futuro, migraciones periódicas (ej. cada 3 meses) hacia almacenamiento frío sin comprometer la integridad de la base de datos central.

6. Infraestructura y Buenas Prácticas de Código

6.1. Dependencia de Almacenamiento Local (Fallback) y Portabilidad

- **Observación:** El Backend utiliza almacenamiento en volumen **/data** cuando está disponible, pero mantiene un fallback a directorios locales dentro del contenedor si el volumen no está montado/configurado (app_railway.py).
- **Impacto:** Ante una mala configuración del despliegue o cambios de infraestructura, el sistema podría operar en modo “local” y exponer riesgo de pérdida de archivos al reciclar contenedores, además de dificultar portabilidad entre entornos.
- **Recomendación:** En ambientes productivos, eliminar o deshabilitar el fallback a almacenamiento local dentro del contenedor; es preferible fallar explícitamente (error de configuración) a operar en un modo que pueda dejar documentos en una ruta no persistente. Asegurar que el despliegue productivo utilice almacenamiento persistente (volumen/bind mount o equivalente).

6.2. Enlaces Rígidos (Hardcoding) en el Frontend

- **Observación:** Se identifican múltiples referencias donde la URL base de la API (ej. https://algarrobobase2...) se encuentra incrustada rígidamente en HTML y JavaScript (hardcoded), en lugar de resolverse desde una configuración central por entorno.
- **Impacto:** Riesgo de fractura del sistema durante la migración al servidor municipal / cambio de dominio o reverse proxy, obligando a buscar y reemplazar manualmente múltiples ocurrencias y aumentando la probabilidad de errores/regresiones.
- **Recomendación:** Centralizar la URL base en un único archivo global de configuración consumido por todas las vistas, como ya se hace con **API_CONFIG.BASE_URL** (solo falta consumirla correctamente), o inyectarla directamente desde las variables de entorno del servidor Backend hacia las plantillas.

6.3. Borrado Físico de Usuarios (Error 500 y Riesgo de Integridad)

- **Observación:** Desde la interfaz administrativa, la eliminación de usuarios genera errores internos (HTTP 500). El endpoint de eliminación intenta un borrado físico (**DELETE FROM users**) (users_routes.py).
- **Impacto:** El borrado físico es incompatible con la conservación de historial y, en presencia de relaciones referenciales (ej. usuarios referenciados por proyectos/auditoría), puede gatillar abortos de transacción por integridad referencial, dejando la operación inutilizable y generando inconsistencias operativas.
- **Recomendación:** En producción, deshabilitar el borrado físico como operación estándar y estandarizar el borrado lógico mediante **activo=false** (manteniendo el identificador para preservar integridad y trazabilidad). Permitir borrado físico solo con excepción controlada para usuarios sin dependencias, exigiendo confirmación explícita de administrador y un motivo obligatorio de eliminación, registrándolo en auditoría/control de actividad.

6.4. Observaciones Menores (UI/UX)

- **Acceso a Chatbot IA no disponible para admin_proyectos**
    - **Observación:** La navegación construye la ruta de Chat en función del rol (**baseDir + "chat.html"**), por lo que para **admin_proyectos** apunta a **/.../admin_proyectos/chat.html** (**layout.js**). En el árbol actual solo existe **chat.html** para **admin_general**, por lo que el rol **admin_proyectos** no puede acceder.
    - **Impacto:** Inconsistencia funcional (módulo visible/esperable vs no implementado) y fricción operativa.
    - **Recomendación:** Implementar la vista equivalente para ese rol.

- **Enlace incorrecto desde inicio de administración hacia "Proyectos/SECPLAC"**
    - **Observación:** En la página de inicio de administración, el acceso a SECPLAC apunta a **../division/secplan/admin_general/proyecto.html** en vez de la ruta correspondiente a admin_proyectos.
    - **Impacto:** Navegación confusa y logout inesperado (de acuerdo a los últimos cambios vistos).
    - **Recomendación:** Corregir el enlace o generarlo dinámicamente según el rol del usuario (alineado con la lógica de **layout.js**).

7. Inconsistencias en la Documentación Técnica

7.1. Discrepancia en el Almacenamiento de Sesiones (JWT)

- **Observación:** La documentación indica autenticación JWT almacenada en cookies seguras (HttpOnly/SameSite). Sin embargo, la implementación efectiva opera principalmente mediante token enviado en el header **Authorization** desde el Frontend, el cual obtiene el token desde **localStorage**. Si bien el Backend setea una cookie **authToken** en login, los decoradores de autenticación no consumen cookies actualmente (solo header/querystring). Referencias: **README.md**, **auth_routes.py**, **decorators.py**, **api.js**.
- **Impacto Operativo y de Seguridad:** la divergencia dificulta despliegues y soporte (no queda 100% claro el mecanismo "oficial"). En seguridad, **localStorage** eleva la exposición ante XSS en comparación con cookies HttpOnly; por otro lado, cookies cross-site requieren configuración cuidadosa (SameSite, CORS, credentials) y puede no funcionar con Front/Back en dominios distintos.
- **Recomendación:** Actualizar la documentación para reflejar el flujo real (Bearer token en header) o, alternativamente, completar el diseño basado en cookies (incluyendo consumo server-side y configuración cross-site si aplica). En ambos casos, dejar explícito el mecanismo soportado en producción.

8. Consideraciones sobre el Alcance de la Refactorización

Nota Metodológica: Los hallazgos, vulnerabilidades y deudas técnicas detallados en las secciones anteriores representan patrones críticos detectados durante la auditoría. Sin embargo, las ubicaciones de código y archivos mencionados sirven como ejemplos evidenciales y no constituyen una lista exhaustiva de cada línea de código defectuosa.

Se advierte explícitamente al equipo de desarrollo que la corrección no debe limitarse a parchear únicamente las líneas citadas en este informe. Se les incentiva a utilizar

estos hallazgos como base para realizar una revisión cruzada y una refactorización integral en todos los módulos del sistema que compartan la misma lógica defectuosa (por ejemplo: rastrear todas las variables de entorno expuestas, y todos los endpoints que carezcan de protección estricta de roles).

9. Conclusión General

Esta segunda fase de auditoría técnica confirma que la plataforma Geoportal Municipal ha logrado avances significativos en la reducción de su deuda técnica con respecto al proceso de revisión anterior, mostrando mejoras valorables en sus capas de infraestructura, base de datos y modelo de seguridad. Sin embargo, aún existen brechas estructurales que deben ser subsanadas para poder considerar la entrega como completa.

La delegación del control de acceso al Frontend (vulnerando el Principio de Menor Privilegio), la presencia de credenciales críticas expuestas en el código fuente, la falta de atomicidad en las transacciones de auditoría y la persistencia de redundancia en tablas de legado, sumado al diseño de procesos síncronos bloqueantes que comprometen la estabilidad del sistema bajo carga concurrente, evidencian que el software aún requiere un ciclo de maduración y refinamiento técnico antes de considerar viable su paso a producción.

Para garantizar el éxito y la sostenibilidad del proyecto, se sugiere priorizar la revisión y el ajuste de los hallazgos documentados en Seguridad (Sección 3), Rendimiento (Sección 4) y Arquitectura (Sección 5). El objetivo principal de esta iteración es asegurar la recepción de un sistema resiliente, limpio y seguro, garantizando así la continuidad operativa y el resguardo íntegro de información institucional a largo plazo.